

Modelos Avanzados de Computación

Relación 4 — Resolución para estudio

Nota. Documento de estudio con las soluciones de la Relación 4. La relación cubre clases de complejidad (P , NP , $CoNP$, $L = ESPACIO(\log n)$, $ESPACIO(n)$), reducciones en espacio logarítmico, grafos y composición de funciones. Las técnicas empleadas (verificadores con certificado, simulación con punteros en espacio $O(\log n)$, argumentos de cierre por unión/intersección, diagonalización implícita vía teoremas de jerarquía) son resultados clásicos del área, y coinciden con los desarrollos estándar de Sipser (*Introduction to the Theory of Computation*), Papadimitriou (*Computational Complexity*) y Garey–Johnson (*Computers and Intractability*). Donde un argumento sigue de cerca uno de estos textos se indica explícitamente.

Índice

1	Grafos dirigidos acíclicos (DAG)	1
2	Grafos bipartitos	3
3	P es cerrada para unión e intersección	4
4	Cierre de clases de funciones por composición polinómica	4
5	$L = \{wcw : w \in \{0, 1\}^*\}$ se reconoce en espacio $\log n$	6
6	Si $L \in P$ entonces $L^* \in P$	7
7	Si $L \in NP$ entonces $L^* \in NP$	7
8	NP es cerrada para unión e intersección	8
9	Si $NP \neq CoNP$ entonces $P \neq NP$	9
10	Paréntesis bien emparejados está en L	9
11	Dos tipos de paréntesis	10
12	El palíndromo requiere $O(n)$ sin retroceder	11
13	$NP \neq ESPACIO(n)$	11

1 Grafos dirigidos acíclicos (DAG)

(a) Todo DAG finito tiene una fuente

Proposición 1. *Todo grafo dirigido finito y acíclico posee al menos una fuente: un nodo sin arcos entrantes.*

Demostración. Razonamos por contradicción. Sea $G = (V, E)$ finito y acíclico, y supongamos que *ningún* nodo es fuente; es decir, todo nodo tiene al menos un arco entrante.

Construimos un camino hacia atrás. Tomamos un nodo cualquiera v_0 . Como v_0 no es fuente, existe un arco $(v_1, v_0) \in E$. Como v_1 tampoco es fuente, existe $(v_2, v_1) \in E$. Repitiendo, generamos una sucesión

$$\cdots \rightarrow v_3 \rightarrow v_2 \rightarrow v_1 \rightarrow v_0,$$

en la que cada v_{i+1} tiene un arco hacia v_i . Como $|V| = n$ es finito, entre los $n + 1$ primeros nodos $v_0, v_1, \dots, v_n$ debe haber, por el principio del palomar, dos índices $i < j$ con $v_i = v_j$. El tramo

$$v_j \rightarrow v_{j-1} \rightarrow \cdots \rightarrow v_i = v_j$$

es entonces un **ciclo dirigido**, lo que contradice que G sea acíclico.

Por tanto, algún nodo no puede tener arco entrante: G tiene una fuente. ■

Nota. Por simetría (invirtiendo todos los arcos) el mismo argumento prueba que todo DAG finito tiene también un *sumidero* (nodo sin arcos salientes). Esto se usará implícitamente para el orden topológico.

(b) Numeración topológica

Proposición 2. *Un grafo dirigido con n nodos es acíclico si y solo si se pueden numerar sus nodos $1, 2, \dots, n$ de modo que todo arco vaya de un número menor a uno mayor.*

Demostración. ($\Leftarrow$) Supongamos que existe tal numeración $\nu : V \rightarrow \{1, \dots, n\}$ con $\nu(u) < \nu(v)$ para todo arco (u, v) . Si hubiera un ciclo $w_1 \rightarrow w_2 \rightarrow \cdots \rightarrow w_k \rightarrow w_1$, encadenando las desigualdades obtendríamos

$$\nu(w_1) < \nu(w_2) < \cdots < \nu(w_k) < \nu(w_1),$$

es decir $\nu(w_1) < \nu(w_1)$, absurdo. Luego G es acíclico.

($\Rightarrow$) Supongamos G acíclico. Construimos la numeración por inducción sobre n (este es el algoritmo de *ordenación topológica*).

Por el apartado (a), G tiene una fuente s . Le asignamos el número 1: $\nu(s) = 1$. Consideramos $G' = G \setminus \{s\}$ (eliminamos s y sus arcos salientes). G' sigue siendo acíclico (quitar nodos/arcos no crea ciclos) y tiene $n - 1$ nodos; por hipótesis de inducción admite una numeración $\nu' : V \setminus \{s\} \rightarrow \{1, \dots, n - 1\}$ compatible. Definimos $\nu(v) = \nu'(v) + 1$ para $v \neq s$.

Comprobamos que todo arco va de menor a mayor:

- Arcos que salen de s : van de $\nu(s) = 1$ a algún $\nu(v) \geq 2$. Correcto.
- Arcos que no tocan s : estaban en G' y eran crecientes para ν' , luego siguen siendo crecientes para $\nu = \nu' + 1$. Correcto.
- No hay arcos *entrantes* a s porque s es fuente.

Esto completa la inducción. ■

(c) Algoritmo polinómico para decidir si un grafo es acíclico

La prueba de (b) es constructiva y da directamente el algoritmo (ordenación topológica mediante eliminación iterativa de fuentes, conocido como algoritmo de Kahn):

Algoritmo ESACICLICO($G = (V, E)$):

1. Calcular el grado de entrada $d^-(v)$ de cada nodo. Inicializar una cola Q con todos los nodos de grado de entrada 0 (las fuentes). Contador $c \leftarrow 0$.
2. Mientras $Q \neq \emptyset$:
 - (a) Extraer u de Q ; $c \leftarrow c + 1$.
 - (b) Para cada arco (u, w) : decrementar $d^-(w)$; si queda en 0, añadir w a Q .
3. Si $c = |V|$ devolver ACICLICO; en otro caso devolver TIENE CICLO.

Corrección. Si G es acíclico, por (a) en cada paso del subgrafo restante siempre queda una fuente, así que el proceso elimina los n nodos y $c = n$. Recíprocamente, si el proceso se detiene con $c < n$, todos los nodos restantes tienen grado de entrada ≥ 1 , luego (por el argumento de (a) aplicado al subgrafo restante) ese subgrafo contiene un ciclo, y por tanto G también.

Complejidad. Cada arco se procesa exactamente una vez (al eliminar su origen) y cada nodo se encola/desencola una vez. El coste es $O(|V| + |E|)$, lineal y en particular polinómico.

2 Grafos bipartitos

(a) Bipartito $\Leftrightarrow$ todos los ciclos de longitud par

Trabajamos con grafos *no dirigidos*.

Proposición 3. Un grafo $G = (V, E)$ es bipartito si y solo si todos sus ciclos tienen longitud par.

Demostración. ($\Rightarrow$) Supongamos G bipartito con partición $V = A \dot{\cup} B$ (toda arista une un vértice de A con uno de B). Sea $v_0, v_1, \dots, v_k = v_0$ un ciclo. Cada arista cambia de lado: si $v_0 \in A$ entonces $v_1 \in B$, $v_2 \in A$, $\dots$; en general $v_i \in A$ si i es par y $v_i \in B$ si i es impar. Como debe cerrarse en $v_k = v_0 \in A$, el índice k ha de ser par. Luego todo ciclo tiene longitud par.

($\Leftarrow$) Supongamos que todos los ciclos son de longitud par. Basta trabajar por componentes conexas (la unión de biparticiones de cada componente es una bipartición del grafo). Fijada una componente, elegimos una raíz r y definimos, para cada vértice v , la distancia $d(v)$ = longitud del camino más corto de r a v . Coloreamos:

$$A = \{v : d(v) \text{ par}\}, \quad B = \{v : d(v) \text{ impar}\}.$$

Veamos que ninguna arista une dos vértices del mismo color. Sea $\{u, v\} \in E$. Si fuera $d(u) \equiv d(v) \pmod{2}$, consideremos los caminos más cortos P_u de r a u y P_v de r a v . Sea w su último vértice común. Los tramos $w \rightarrow u$ y $w \rightarrow v$ tienen longitudes $d(u) - d(w)$ y $d(v) - d(w)$, de la *misma* paridad. El recorrido

$$w \rightarrow \dots \rightarrow u - v \rightarrow \dots \rightarrow w$$

(bajar a u , usar la arista $\{u, v\}$, subir desde v) es un ciclo de longitud

$$(d(u) - d(w)) + 1 + (d(v) - d(w)),$$

que es **impar** (suma de dos números de igual paridad más 1). Esto contradice la hipótesis. Por tanto toda arista une colores distintos y la componente es bipartita. ■

(b) Algoritmo polinómico para comprobar bipartición

La prueba sugiere directamente un algoritmo de *2-coloreado* por BFS:

Algoritmo ESBIPARTITO(G):

1. Marcar todos los vértices como sin color.
2. Para cada vértice r aún sin color (cabecera de una componente):
 - (a) Asignar $color(r) = 0$ y lanzar un BFS desde r .
 - (b) Al visitar una arista $\{u, v\}$: si v no tiene color, asignar $color(v) = 1 - color(u)$; si ya lo tiene y $color(v) = color(u)$, devolver NO BIPARTITO.
3. Si nunca se produjo conflicto, devolver BIPARTITO con la partición dada por los colores.

Corrección. El BFS asigna a cada vértice la paridad de su distancia a la raíz, exactamente el coloreado de la prueba de (a). Un conflicto ($color(u) = color(v)$ en una arista) corresponde a la situación del ciclo impar; su ausencia certifica una bipartición válida.

Complejidad. Es un recorrido BFS estándar: $O(|V| + |E|)$, polinómico.

3 P es cerrada para unión e intersección

Teorema 1. Si $L_1, L_2 \in P$, entonces $L_1 \cup L_2 \in P$ y $L_1 \cap L_2 \in P$.

Demostración. Por hipótesis existen máquinas de Turing deterministas M_1 y M_2 que deciden L_1 y L_2 en tiempo $O(n^{c_1})$ y $O(n^{c_2})$ respectivamente, para ciertas constantes c_1, c_2 .

Construimos una máquina M que, con entrada x de longitud n :

1. Simula $M_1(x)$ y guarda su respuesta $b_1 \in \{\text{Sí}, \text{No}\}$.
2. Simula $M_2(x)$ y guarda su respuesta b_2 .
3. **Unión:** acepta si $b_1 = \text{Sí}$ o $b_2 = \text{Sí}$.

Intersección: acepta si $b_1 = \text{Sí}$ y $b_2 = \text{Sí}$.

M es determinista y siempre se detiene. Su tiempo es a lo sumo

$$O(n^{c_1}) + O(n^{c_2}) + O(1) = O(n^{\max(c_1, c_2)}),$$

es decir, polinómico. Además $L(M) = L_1 \cup L_2$ (resp. $L_1 \cap L_2$) por construcción. Luego ambos lenguajes están en P . ■

Nota. El mismo argumento prueba que P es cerrada para complementación: basta intercambiar los estados de aceptación y rechazo de una máquina polinómica determinista (la determinación y la parada garantizada son esenciales aquí).

4 Cierre de clases de funciones por composición polinómica

Definición 1 (recordatorio del enunciado). Sea $\mathcal{C}$ una clase de funciones $\mathbb{N} \rightarrow \mathbb{N}$ y p un polinomio arbitrario.

- $\mathcal{C}$ es cerrada para la composición polinómica **por la izquierda** si $f \in \mathcal{C} \Rightarrow p(f(n)) = O(g(n))$ para alguna $g \in \mathcal{C}$.
- $\mathcal{C}$ es cerrada para la composición polinómica **por la derecha** si $f \in \mathcal{C} \Rightarrow f(p(n)) = O(g(n))$ para alguna $g \in \mathcal{C}$.

Como indica la nota del enunciado, por tratarse de órdenes de complejidad basta considerar $p(n) = n^l$ con $l \in \mathbb{N}$.

Idea. Por la izquierda aplicamos primero f y luego elevamos a una potencia: $p(f(n)) = f(n)^l$. Por la derecha sustituimos n por n^l dentro de f : $f(n^l)$. La clase es cerrada si el resultado vuelve a estar dominado por una función de la misma familia.

A continuación analizamos las seis familias. Resumimos al final en una tabla.

(a) $\mathcal{C} = \{n^k : k > 0\}$ (polinomios monómicos)

Sea $f(n) = n^k \in \mathcal{C}$.

Izquierda. $p(f(n)) = (n^k)^l = n^{kl}$. Como $kl > 0$, la función $n^{kl} \in \mathcal{C}$, y $p(f(n)) = O(n^{kl})$.
Cerrada por la izquierda.

Derecha. $f(p(n)) = (n^l)^k = n^{lk} \in \mathcal{C}$. **Cerrada por la derecha.**

(a) cerrada por ambos lados.

(b) $\mathcal{C} = \{n \cdot k : k > 0\}$ (funciones lineales)

Aquí $f(n) = kn$ es lineal con pendiente $k > 0$ (el factor constante, no el exponente).

Izquierda. $p(f(n)) = (kn)^l = k^l n^l$. Para $l \geq 2$ esto es $\Theta(n^l)$, un crecimiento *superlineal* que **no** puede acotarse por ninguna función lineal $g(n) = k'n$ (pues $n^l/n = n^{l-1} \rightarrow \infty$). **No cerrada por la izquierda.**

Derecha. $f(p(n)) = k(n^l) = kn^l$. Igual que antes, para $l \geq 2$ es $\Theta(n^l)$, no acotable por una función lineal. **No cerrada por la derecha.**

(b) no cerrada por ningún lado.

(c) $\mathcal{C} = \{k^n : k > 0\}$ (exponenciales de base constante)

Sea $f(n) = k^n$. (Tomamos $k \geq 2$; para $k = 1$ es la constante 1 y para $k > 1$ el análisis es el interesante.)

Izquierda. $p(f(n)) = (k^n)^l = k^{ln} = (k^l)^n$. Llamando $k' = k^l > 0$ obtenemos $(k')^n \in \mathcal{C}$. **Cerrada por la izquierda.**

Derecha. $f(p(n)) = k^{n^l}$. Para $l \geq 2$ esto crece como k^{n^2} , mucho más rápido que cualquier $(k')^n = k'^n$, ya que

$$\frac{k^{n^l}}{(k')^n} = \frac{k^{n^l}}{k^{n \log_k k'}} = k^{n^l - n \log_k k'} \rightarrow \infty.$$

No existe $g \in \mathcal{C}$ que lo domine. **No cerrada por la derecha.**

(c) cerrada por la izquierda, no por la derecha.

(d) $\mathcal{C} = \{2^{n^k} : k > 0\}$ (exponenciales con exponente polinómico)

Sea $f(n) = 2^{n^k}$.

Izquierda. $p(f(n)) = (2^{n^k})^l = 2^{ln^k}$. Necesitamos acotar 2^{ln^k} por algún $2^{n^{k'}}$. Como l es constante, para n grande $ln^k \leq n^{k+1}$, luego

$$2^{ln^k} \leq 2^{n^{k+1}} = O(2^{n^{k+1}}), \quad 2^{n^{k+1}} \in \mathcal{C}.$$

Cerrada por la izquierda.

Derecha. $f(p(n)) = 2^{(n^l)^k} = 2^{n^{lk}}$. Como $lk > 0$, la función $2^{n^{lk}} \in \mathcal{C}$. **Cerrada por la derecha.**

(d) cerrada por ambos lados.

(e) $\mathcal{C} = \{\log^k(n) : k > 0\}$ (potencias de logaritmo)

Sea $f(n) = (\log n)^k = \log^k n$ (con redondeo al entero más cercano, irrelevante para el orden).

Izquierda. $p(f(n)) = (\log^k n)^l = \log^{kl} n \in \mathcal{C}$, pues $kl > 0$. **Cerrada por la izquierda.**

Derecha. $f(p(n)) = \log^k(n^l) = (l \log n)^k = l^k \log^k n = O(\log^k n)$, y $\log^k n \in \mathcal{C}$. **Cerrada por la derecha.**

(e) cerrada por ambos lados.

(f) $\mathcal{C} = \{\log n\}$ (un único elemento: el logaritmo)

Ahora la clase tiene *una sola* función, $f(n) = \log n$, así que la única candidata posible para g es la propia $\log n$.

Izquierda. $p(f(n)) = (\log n)^l = \log^l n$. Para $l \geq 2$ se tiene $\log^l n / \log n = \log^{l-1} n \rightarrow \infty$, de modo que $\log^l n \neq O(\log n)$. La única g disponible no lo acota. **No cerrada por la izquierda.**

Derecha. $f(p(n)) = \log(n^l) = l \log n = O(\log n)$, y la única función de la clase, $\log n$, sí lo domina. **Cerrada por la derecha.**

(f) cerrada por la derecha, no por la izquierda.

Resumen

Clase	Izquierda $p(f(n)) = f(n)^l$	Derecha $f(n^l)$
(a) $\{n^k\}$	Sí	Sí
(b) $\{k n\}$	No	No
(c) $\{k^n\}$	Sí	No
(d) $\{2^{n^k}\}$	Sí	Sí
(e) $\{\log^k n\}$	Sí	Sí
(f) $\{\log n\}$	No	Sí

Nota. Patrón conceptual: la composición *por la izquierda* eleva f a la potencia l ; sobrevive cuando la clase es estable al elevar a potencias constantes (familias paramétricas en el exponente o en el índice de logaritmo). La composición *por la derecha* mete n^l dentro de f ; sobrevive cuando “elevar el argumento a una potencia” equivale a moverse dentro de la familia. Las familias con *un solo grado de libertad mal situado* (lineales, o el logaritmo aislado) fallan en el lado que ataca ese grado de libertad.

5 $L = \{wcw : w \in \{0, 1\}^*\}$ se reconoce en espacio $\log n$

Teorema 2. *El lenguaje $L = \{wcw : w \in \{0, 1\}^*\}$ pertenece a $L = \text{ESPACIO}(\log n)$.*

Idea. No podemos copiar w a la cinta de trabajo (eso costaría espacio lineal). En su lugar usamos *punteros*: comparamos las dos mitades posición a posición guardando solo índices, y cada índice cabe en $O(\log n)$ bits.

Demostración. Trabajamos con el modelo estándar de máquina de Turing con cinta de entrada de *solo lectura* y una cinta de trabajo aparte cuyo espacio es el que contamos. Sea la entrada de longitud n .

Fase 1: validar el formato. Recorremos la entrada y comprobamos que contiene *exactamente* un símbolo c . Para ello mantenemos un contador del número de c vistos y la posición m de la c . El contador y la posición son números $\leq n$, luego ocupan $O(\log n)$ bits. Si el número de c no es 1, rechazar. Sea m la posición de la c (separa la entrada en parte izquierda $u = x_1 \cdots x_{m-1}$ y parte derecha $v = x_{m+1} \cdots x_n$).

Fase 2: comprobar longitudes iguales. La pertenencia exige $|u| = |v|$, es decir $m - 1 = n - m$. Esto se verifica con aritmética sobre los índices m y n , de nuevo en $O(\log n)$ espacio. Si no coinciden, rechazar.

Fase 3: comparar carácter a carácter. Para $i = 1, 2, \dots, m - 1$ comparamos el símbolo en la posición i (mitad izquierda) con el de la posición $m + i$ (mitad derecha). Concretamente, mantenemos un contador i y, en cada iteración:

- movemos la cabeza de *solo lectura* a la posición i y leemos x_i (guardando ese único bit);
- movemos la cabeza a la posición $m + i$ y leemos x_{m+i} ;
- si $x_i \neq x_{m+i}$, rechazar.

Si todas las comparaciones coinciden, aceptar.

Análisis de espacio. En la cinta de trabajo solo almacenamos en cada momento un contador (el índice i , o m , o el número de c) y a lo sumo uno o dos bits leídos. Todos los índices están acotados por n , luego se codifican en $\lceil \log_2 n \rceil + O(1)$ bits. El número de variables es constante. Por tanto el espacio total es $O(\log n)$.

(El tiempo es polinómico — $O(n^2)$ por los reposicionamientos de la cabeza — pero el enunciado solo exige la cota de espacio.) ■

6 Si $L \in P$ entonces $L^* \in P$

Teorema 3. P es cerrada para la estrella de Kleene: si $L \in P$, entonces $L^* \in P$.

Idea. $x \in L^*$ significa que x se puede *partir* en bloques consecutivos $x = w_1 w_2 \cdots w_k$ con cada $w_j \in L$ (y $\varepsilon \in L^*$ siempre). Esto se decide con programación dinámica sobre los prefijos de x .

Demostración. Sea M una máquina que decide L en tiempo $O(n^c)$. Dada $x = x_1 x_2 \cdots x_n$, escribimos $x[i..j] = x_i \cdots x_j$ (subcadena), y por convenio $x[i..i-1] = \varepsilon$.

Definimos un vector booleano $A[0..n]$ con la interpretación

$$A[j] = \text{Verdadero} \iff \text{el prefijo } x[1..j] \in L^*.$$

Recurrencia (descomponiendo según dónde empieza el último bloque):

$$A[0] = \text{Verdadero (el prefijo vacío está en } L^*),$$

$$A[j] = \bigvee_{0 \leq i < j} \left(A[i] \wedge (x[i+1..j] \in L) \right), \quad 1 \leq j \leq n.$$

Es decir, $x[1..j] \in L^*$ si existe un punto de corte i tal que el prefijo $x[1..i]$ ya estaba en L^* y el último trozo $x[i+1..j]$ pertenece a L .

Algoritmo. Calcular $A[0], A[1], \dots, A[n]$ en orden creciente usando la recurrencia; para cada par (i, j) se invoca M sobre la subcadena $x[i+1..j]$. Aceptar si $A[n] = \text{Verdadero}$.

Corrección. Por inducción sobre j : $A[j]$ es verdadero exactamente cuando $x[1..j]$ admite una descomposición en factores de L , que es justo la definición de pertenencia a L^* . El caso base $A[0]$ corresponde a $\varepsilon \in L^*$. En particular $A[n]$ decide $x \in L^*$.

Complejidad. Hay $O(n^2)$ pares (i, j) . Cada uno requiere una llamada a M sobre una subcadena de longitud $\leq n$, con coste $O(n^c)$. El total es

$$O(n^2) \cdot O(n^c) = O(n^{c+2}),$$

polinómico. Por tanto $L^* \in P$. ■

7 Si $L \in NP$ entonces $L^* \in NP$

Teorema 4. NP es cerrada para la estrella de Kleene: si $L \in NP$, entonces $L^* \in NP$.

Idea. Caracterizamos NP por verificador y certificado. Para L^* el certificado consta de: (i) los *puntos de corte* que parten x en bloques, y (ii) un certificado de pertenencia a L para cada bloque. Todo ello es de tamaño polinómico y verificable en tiempo polinómico.

Demostración (versión verificador). Como $L \in \text{NP}$, existe un verificador determinista V y un polinomio q tales que, para toda cadena w ,

$$w \in L \iff \exists c_w, |c_w| \leq q(|w|), \quad V(w, c_w) = 1,$$

y V corre en tiempo polinómico.

Definimos un verificador V^* para L^* . Dado x de longitud n , el certificado propuesto es

$$(i_0 = 0 < i_1 < i_2 < \dots < i_k = n, \quad c_1, c_2, \dots, c_k),$$

donde los i_j marcan los cortes (el bloque j -ésimo es $w_j = x[i_{j-1} + 1..i_j]$) y c_j es un certificado de que $w_j \in L$. El caso $x = \varepsilon$ se acepta directamente ($k = 0$).

V^* procede así:

1. Comprobar que $0 = i_0 < i_1 < \dots < i_k = n$ (los cortes son válidos y cubren todo x sin solapes).
2. Para cada $j = 1, \dots, k$: extraer $w_j = x[i_{j-1} + 1..i_j]$ y verificar $V(w_j, c_j) = 1$.
3. Aceptar si y solo si todas las verificaciones tienen éxito.

Corrección. $x \in L^*$ si y solo si existe una partición de x en bloques $w_1 \dots w_k$ con cada $w_j \in L$, lo cual equivale a la existencia de cortes válidos y de certificados c_j aceptados por V . Esto es exactamente lo que V^* comprueba.

Tamaño del certificado. Hay a lo sumo $k \leq n$ cortes, cada uno un índice $\leq n$: $O(n \log n)$ bits. Cada c_j tiene tamaño $\leq q(|w_j|) \leq q(n)$, y $\sum_j |w_j| = n$, luego la suma de longitudes de los certificados es a lo sumo $n \cdot q(n)$, polinómica.

Tiempo de verificación. Comprobar los cortes es lineal; ejecutar V sobre cada bloque cuesta polinómico y se hace $k \leq n$ veces. Total polinómico.

Por tanto $L^* \in \text{NP}$. ■

Nota (demostración alternativa con MT no determinista). De forma equivalente: una máquina no determinista para L^* va leyendo x de izquierda a derecha, “adivina” dónde termina cada bloque, y para cada bloque ejecuta (no deterministamente) la máquina de L . Como $\sum |w_j| = n$, el cómputo total es de longitud polinómica. También se puede usar la técnica de programación dinámica del Ejercicio 6 sustituyendo “ $x[i + 1..j] \in L$ ” por una llamada al verificador no determinista; el esquema es idéntico.

8 NP es cerrada para unión e intersección

Teorema 5. Si $L_1, L_2 \in \text{NP}$, entonces $L_1 \cup L_2 \in \text{NP}$ y $L_1 \cap L_2 \in \text{NP}$.

Demostración. Usamos la caracterización por verificadores. Sean V_1, V_2 verificadores polinómicos y q_1, q_2 polinomios con

$$x \in L_i \iff \exists c_i, |c_i| \leq q_i(|x|), \quad V_i(x, c_i) = 1 \quad (i = 1, 2).$$

Unión. El certificado es un par (etiqueta, testigo): un bit $b \in \{1, 2\}$ que indica “a qué lenguaje pertenece x ” junto con el certificado correspondiente c_b . El verificador $V_\cup(x, (b, c))$ ejecuta $V_b(x, c)$ y acepta si este acepta. Entonces

$$x \in L_1 \cup L_2 \iff (\exists c_1 : V_1(x, c_1) = 1) \text{ o } (\exists c_2 : V_2(x, c_2) = 1) \iff \exists (b, c) : V_\cup(x, (b, c)) = 1.$$

El certificado tiene tamaño $1 + \max(q_1, q_2)(|x|)$ y $V_\cup$ es polinómico. Luego $L_1 \cup L_2 \in \text{NP}$.

Intersección. El certificado es el *par* (c_1, c_2) de ambos testigos. El verificador $V_{\cap}(x, (c_1, c_2))$ ejecuta $V_1(x, c_1)$ y $V_2(x, c_2)$ y acepta si *ambos* aceptan. Entonces

$$x \in L_1 \cap L_2 \iff (\exists c_1 : V_1(x, c_1) = 1) \text{ y } (\exists c_2 : V_2(x, c_2) = 1) \iff \exists (c_1, c_2) : V_{\cap}(x, (c_1, c_2)) = 1.$$

Un detalle sobre la intersección: el cuantificador conjunto “ $\exists(c_1, c_2)$ ” es correcto porque los dos testigos son *independientes* — elegir uno no condiciona al otro — de modo que “ $\exists c_1 \exists c_2$ ” equivale a “existe el par”. El certificado tiene tamaño $q_1(|x|) + q_2(|x|)$, polinómico, y $V_{\cap}$ es polinómico. Luego $L_1 \cap L_2 \in \text{NP}$. ■

9 Si $\text{NP} \neq \text{CoNP}$ entonces $\text{P} \neq \text{NP}$

Teorema 6. Si $\text{NP} \neq \text{CoNP}$, entonces $\text{P} \neq \text{NP}$.

Idea. Demostramos el contrarrecíproco: si $\text{P} = \text{NP}$ entonces $\text{NP} = \text{CoNP}$. La clave es que P es cerrada para complementación (Ejercicio 3 / Nota), propiedad que *no* se sabe que tenga NP .

Demostración. Probamos la afirmación equivalente (contrarrecíproca):

$$\text{P} = \text{NP} \implies \text{NP} = \text{CoNP}.$$

Supongamos $\text{P} = \text{NP}$. Recordemos que P es cerrada para complementación: si $A \in \text{P}$, intercambiando aceptación y rechazo en una máquina determinista polinómica que decide A obtenemos otra que decide $\bar{A}$ en tiempo polinómico, luego $\bar{A} \in \text{P}$.

Veamos la doble inclusión $\text{NP} = \text{CoNP}$.

$\text{NP} \subseteq \text{CoNP}$: Sea $A \in \text{NP}$. Por hipótesis $A \in \text{P}$. Como P es cerrada para complemento, $\bar{A} \in \text{P} = \text{NP}$. Pero “ $\bar{A} \in \text{NP}$ ” es precisamente la definición de “ $A \in \text{CoNP}$ ”. Luego $A \in \text{CoNP}$.

$\text{CoNP} \subseteq \text{NP}$: Sea $A \in \text{CoNP}$, esto es $\bar{A} \in \text{NP}$. Por hipótesis $\bar{A} \in \text{P}$, y por cierre bajo complemento $A \in \text{P} = \text{NP}$. Luego $A \in \text{NP}$.

De ambas inclusiones, $\text{NP} = \text{CoNP}$.

Hemos probado $\text{P} = \text{NP} \implies \text{NP} = \text{CoNP}$. Tomando contrarrecíproco:

$$\text{NP} \neq \text{CoNP} \implies \text{P} \neq \text{NP}. \quad \blacksquare$$

10 Paréntesis bien emparejados está en L

Teorema 7. El lenguaje de las cadenas de paréntesis $\{(,)\}$ correctamente emparejados y anidados pertenece a $\text{L} = \text{ESPACIO}(\log n)$.

Idea. Con un *único* tipo de paréntesis no hace falta una pila: basta un *contador* de profundidad. La condición de buen emparejamiento es local: “el contador nunca se vuelve negativo y termina en cero”.

Demostración. Sea la entrada $x = x_1 \cdots x_n$ sobre el alfabeto $\{(,)\}$. Mantenemos en la cinta de trabajo un único contador entero b (“balance”), inicializado a 0. Recorremos la entrada de izquierda a derecha:

- si $x_i = ($, hacer $b \leftarrow b + 1$;
- si $x_i =)$, hacer $b \leftarrow b - 1$;
- si en algún momento $b < 0$, **rechazar** (hay un $)$ sin su $($ previo).

Al terminar, **aceptar** si y solo si $b = 0$.

Corrección. Una cadena de un solo tipo de paréntesis está bien formada si en todo prefijo el número de $($ es $\geq$ el de $)$ (nunca se cierra de más) y al final ambos coinciden (todo lo abierto se cierra). El contador b mide exactamente $\#(- \#)$ del prefijo leído; “ $b \geq 0$ siempre” y “ $b = 0$ ”

al final” son justo esas dos condiciones. Así, $((()))$ y $((()))$ son aceptadas, mientras $)() ($ (el balance se hace -1 al primer símbolo) y $((())) ($ (balance llega a -1) son rechazadas.

Espacio. El contador b satisface $0 \leq b \leq n$ en todo instante (mientras no se rechace), luego se almacena en $\lceil \log_2(n+1) \rceil = O(\log n)$ bits. No se usa nada más. Por tanto el algoritmo es $O(\log n)$ en espacio, y el problema está en L. ■

11 Dos tipos de paréntesis

Teorema 8. *El lenguaje de cadenas bien formadas con dos tipos de paréntesis, $(,)$ y $[,]$, también pertenece a L.*

Idea. Con dos tipos importa el *orden* de cierre (anidamiento correcto: $([] ())$ sí, $([]) ($ no). La forma natural de resolverlo es con una pila, pero una pila puede crecer $\Theta(n)$. El truco para mantenernos en $O(\log n)$ es *no almacenar la pila*: para verificar cada cierre, recalculamos cuál es su paréntesis de apertura asociado recorriendo la entrada con contadores.

Demostración. Condición de validez. La cadena es correcta syss:

1. está balanceada *ignorando el tipo* (el contador global de “abiertos menos cerrados” nunca es negativo y acaba en 0); y
2. cada símbolo de cierre case en *tipo* con su apertura asociada (la apertura que lo empareja en el anidamiento).

Emparejamiento sin pila. Para una posición de cierre j , su apertura asociada es la posición $i < j$ tal que el segmento $x[i+1..j-1]$ está balanceado; equivalentemente, i es la última posición $< j$ en la que el balance (contando *cualquier* tipo) tomó el valor (balance en j) justo *antes* de cerrar. La idea operativa, en espacio $O(\log n)$:

Recorremos x con un contador de balance b (como en el Ejercicio 10, comprobando de paso la condición 1). Cada vez que encontramos un cierre en posición j (supongamos que deja el balance en valor $d = b$ tras decrementar), buscamos su apertura: *reiniciamos* un segundo recorrido desde el principio hasta j llevando un contador auxiliar, y localizamos la última posición $i < j$ que es una apertura y desde la cual el sub-balance de $x[i+1..j-1]$ es 0. Comprobamos que x_i y x_j son del mismo tipo (ambos redondos o ambos cuadrados); si no, rechazar.

Todo el almacenamiento son unos pocos contadores acotados por n (la posición j , la posición candidata i , los balances), cada uno $O(\log n)$ bits. No guardamos la pila completa: la “recomputamos” tantas veces como cierres haya. Esto sacrifica tiempo (polinómico, del orden de $O(n^2)$) a cambio de espacio.

Espacio total: $O(\log n)$. Por tanto el problema con dos tipos de paréntesis también está en L.

Segunda parte: lectura de izquierda a derecha sin retroceder requiere $O(n)$.

Si imponemos que la entrada se lee *una sola vez, de izquierda a derecha, sin volver atrás* (modelo *online* / autómatas con memoria pero sin re-lectura), entonces necesitamos recordar la secuencia de aperturas aún no cerradas, en orden: es esencialmente una **pila** de tipos. Justificación (argumento de distinguibilidad / *fooling set*):

Consideremos los 2^m prefijos formados por m aperturas cada una de tipo redondo o cuadrado, p.ej. $u_S = s_1 s_2 \cdots s_m$ con $s_t \in \{ (, [\}$. Para dos prefijos distintos $u_S \neq u_{S'}$ existe una primera posición t donde difieren los tipos. La continuación que cierra en el orden correcto para u_S —es decir, el “espejo” de tipos $\bar{s}_m \cdots \bar{s}_1$ donde $\bar{(} =)$ y $\bar{[} =]$ — completa u_S a una cadena *válida*, pero aplicada a $u_{S'}$ produce un *desajuste de tipo* en alguna posición, dando cadena *inválida*. Por tanto la máquina debe quedar en *estados de memoria distintos* tras leer u_S y $u_{S'}$ (si coincidieran, aceptaría o rechazaría igual ambas continuaciones, contradicción).

Así, hay al menos 2^m configuraciones de memoria distinguibles tras leer m símbolos. Representar 2^m estados distintos exige al menos $\log_2(2^m) = m$ bits de memoria. Como m puede ser $\Theta(n)$, la memoria necesaria es $\Omega(n)$, es decir $O(n)$ (lineal) en el peor caso. ■

Nota. La diferencia con el Ejercicio 10 (un solo tipo) es esencial: con un tipo, la información relevante de la pila es solo su *altura* (un número, $O(\log n)$ bits); con dos o más tipos hay que recordar también la *secuencia de tipos*, que es información de $\Theta(n)$ bits si no se permite re-leer. La cota logarítmica de la primera parte se logra precisamente *gracias* a poder re-leer la entrada (acceso aleatorio de solo lectura), recomputando la pila en vez de almacenarla.

12 El palíndromo requiere $O(n)$ sin retroceder

Teorema 9. *El problema de reconocer palíndromos requiere memoria $\Omega(n)$ (lineal) si la entrada se lee de izquierda a derecha sin poder volver atrás.*

Idea. Mismo principio de *distinguibilidad* que en el Ejercicio 11: exhibimos exponencialmente muchos prefijos que la máquina *tiene* que distinguir, lo que fuerza un número exponencial de configuraciones de memoria y por tanto $\Omega(n)$ bits.

Demostración. Sea $L_{\text{pal}} = \{w : w = w^R\}$ sobre $\{0,1\}$ (donde w^R es w invertida). Supongamos una máquina M que lee la entrada una sola vez, de izquierda a derecha, con memoria de trabajo de tamaño $s(n)$.

Fijado m , consideramos el conjunto de los 2^m prefijos $u \in \{0,1\}^m$. Afirmamos que M debe llegar a configuraciones de memoria *distintas* tras leer dos prefijos distintos $u \neq u'$ (con $|u| = |u'| = m$).

En efecto, supongamos por reducción al absurdo que M alcanza la *misma* configuración de memoria tras leer u y tras leer u' , con $u \neq u'$. Como $u \neq u'$, difieren en alguna posición t : $u_t \neq u'_t$. Construimos la continuación $z = u^R$ (el reverso de u). Entonces:

- La cadena $uz = uu^R$ es un palíndromo (es de la forma ww^R), así que M debe **aceptar** uz .
- La cadena $u'z = u'u^R$ **no** es un palíndromo: su posición t (en la primera mitad) vale u'_t , mientras que la posición simétrica (en la segunda mitad) corresponde a $u_t \neq u'_t$. Luego M debe **rechazar** $u'z$.

Pero tras leer u y u' la configuración de M es la misma *por hipótesis*, y como M no puede volver atrás, su comportamiento al leer la parte restante z es idéntico en ambos casos: aceptaría ambas o rechazaría ambas. Esto contradice que deba aceptar uz y rechazar $u'z$.

Por tanto las 2^m configuraciones tras leer los 2^m prefijos son todas distintas. El número de configuraciones de memoria con s celdas (sobre un alfabeto de trabajo finito de tamaño γ , con la cabeza en una de s posiciones y uno de Q estados de control) es a lo sumo

$$\#\text{config} \leq Q \cdot s \cdot \gamma^s = 2^{O(s)}.$$

Necesitamos $\#\text{config} \geq 2^m$, luego $2^{O(s)} \geq 2^m$, de donde $s = \Omega(m)$. Tomando $m = \lfloor n/2 \rfloor$ (cadenas de longitud $n = 2m$), obtenemos

$$s(n) = \Omega(n).$$

Es decir, el palíndromo requiere memoria *lineal* en el modelo de lectura única sin retroceso. ■

Nota. Compara con el Ejercicio 5: $\{w^R\}$ *sí* está en L porque con acceso de *solo lectura con re-posicionamiento* podemos comparar las dos mitades con punteros ($O(\log n)$). El palíndromo $\{ww^R\}$ también estaría en L con re-lectura (comparando x_i con x_{n+1-i} mediante dos punteros). La cota $\Omega(n)$ de este ejercicio depende *críticamente* de la restricción “sin volver atrás”: es lo que prohíbe usar punteros y obliga a memorizar la primera mitad entera.

13 $\text{NP} \neq \text{ESPACIO}(n)$

Nota. Este ejercicio es delicado. **No** se conoce si $\text{NP} \subseteq \text{ESPACIO}(n)$ o viceversa: lo que se demuestra es que las dos clases *no pueden coincidir*, exhibiendo una *propiedad estructural* que

una tiene y la otra no. Seguimos la sugerencia del enunciado: NP es cerrada bajo reducciones en espacio logarítmico (en general, bajo reducciones de muchos-uno polinómicas/log-espaciales), mientras que ESPACIO(n) *no* lo es. Una clase cerrada bajo log-reducciones y otra que no lo es no pueden ser la misma clase. El ingrediente que rompe el cierre de ESPACIO(n) es el **Teorema de Jerarquía de Espacio**.

Definición 2 (cierre bajo reducciones en L). Una clase $\mathcal{D}$ es *cerrada para reducciones en espacio logarítmico* si: siempre que $P_1 \propto P_2$ (reducción muchos-uno calculable en espacio log) y $P_2 \in \mathcal{D}$, se tiene $P_1 \in \mathcal{D}$.

Paso 1: NP es cerrada para reducciones en L

Lema 1. Si $P_1 \propto P_2$ (reducción en espacio logarítmico) y $P_2 \in \text{NP}$, entonces $P_1 \in \text{NP}$.

Demostración. Sea f la función de reducción, calculable en espacio $O(\log n)$ (y por tanto en tiempo polinómico, pues una máquina log-espacial tiene a lo sumo $2^{O(\log n)} = n^{O(1)}$ configuraciones distintas, luego se detiene en tiempo polinómico; en particular $|f(x)|$ es de tamaño polinómico). La reducción cumple

$$x \in P_1 \iff f(x) \in P_2.$$

Como $P_2 \in \text{NP}$, sea V_2 su verificador polinómico con certificados de tamaño $\leq q(|y|)$.

Construimos un verificador para P_1 : dado x y un certificado candidato c ,

1. calcular $y = f(x)$ (tiempo polinómico);
2. aceptar si $V_2(y, c) = 1$.

Entonces

$$x \in P_1 \iff f(x) \in P_2 \iff \exists c (|c| \leq q(|f(x)|)) : V_2(f(x), c) = 1,$$

y como $|f(x)|$ es polinómico en $|x|$, el tamaño del certificado y el tiempo de verificación son polinómicos en $|x|$. Luego $P_1 \in \text{NP}$. ■

Paso 2: ESPACIO(n) *no* es cerrada para reducciones en L

La idea es exhibir dos lenguajes $A \propto B$ con $B \in \text{ESPACIO}(n)$ pero $A \notin \text{ESPACIO}(n)$. La reducción será un simple “padding” (relleno) que agranda la entrada, y la separación entre “A dentro” y “A fuera” la garantiza el teorema de jerarquía.

Teorema 10 (Jerarquía de espacio; Stearns–Hartmanis–Lewis). Si $s_1(n) = o(s_2(n))$ y s_2 es construible en espacio (y $s_2(n) \geq \log n$), entonces

$$\text{ESPACIO}(s_1(n)) \subsetneq \text{ESPACIO}(s_2(n)),$$

la inclusión es estricta. En particular, para todo $k \geq 1$,

$$\text{ESPACIO}(n) \subsetneq \text{ESPACIO}(n^2) \subsetneq \dots \subsetneq \text{ESPACIO}(n^k),$$

ya que $n = o(n^2)$, etc.

Lema 2. ESPACIO(n) *no* es cerrada para reducciones en espacio logarítmico.

Demostración. Por la jerarquía de espacio, $\text{ESPACIO}(n) \subsetneq \text{ESPACIO}(n^2)$; fijemos un lenguaje testigo

$$A \in \text{ESPACIO}(n^2) \setminus \text{ESPACIO}(n).$$

(A existe por la inclusión estricta: es decidible en espacio cuadrático pero *no* en espacio lineal.)

Definimos su versión “rellenada”. Añadimos un símbolo nuevo $\# \notin \Sigma$ y ponemos

$$B = \{x\#^t : x \in A, t = |x|^2 - |x|\}.$$

Es decir, a cada $x \in A$ le pegamos suficientes símbolos de relleno para que la longitud total pase de $|x|$ a $|x|^2$. Sea $N = |x\#^t| = |x|^2$ la longitud de la cadena rellena.

(i) $B \in \text{ESPACIO}(n)$. Con entrada z de longitud N : comprobamos que z tiene la forma $x\#^t$ con $t = |x|^2 - |x|$ (verificable contando, en espacio $O(\log N)$), extraemos x y ejecutamos el decididor de A , que usa espacio $O(|x|^2)$. Pero $|x|^2 = N$, luego ese espacio es $O(N)$, *lineal en la longitud de la entrada* z . Por tanto $B \in \text{ESPACIO}(N) = \text{ESPACIO}(n)$. (El relleno “diluye” el coste cuadrático en $|x|$ hasta volverlo lineal en N .)

(ii) $A \propto B$ **en espacio logarítmico**. La reducción $f(x) = x\#^{|x|^2-|x|}$ se calcula en espacio $O(\log |x|)$: copiar x a la salida y luego escribir $|x|^2 - |x|$ símbolos $\#$ llevando solo un contador ($O(\log |x|)$ bits). Claramente

$$x \in A \iff f(x) = x\#^t \in B,$$

así que es una reducción muchos-uno válida y log-espacial.

(iii) **Conclusión.** Tenemos $A \propto B$ con $B \in \text{ESPACIO}(n)$. Si $\text{ESPACIO}(n)$ fuera cerrada para reducciones en L, deberíamos concluir $A \in \text{ESPACIO}(n)$. Pero elegimos $A \notin \text{ESPACIO}(n)$. Contradicción. Luego $\text{ESPACIO}(n)$ **no** es cerrada para reducciones en espacio logarítmico. ■

Conclusión: $\text{NP} \neq \text{ESPACIO}(n)$

Demostración. Combinamos los dos pasos:

- NP *sí* es cerrada para reducciones en espacio logarítmico (Paso 1).
- $\text{ESPACIO}(n)$ *no* es cerrada para reducciones en espacio logarítmico (Paso 2).

El cierre para log-reducciones es una propiedad que una clase tiene o no tiene; no puede ser que dos clases *iguales* difieran en ella. Como NP la posee y $\text{ESPACIO}(n)$ no, forzosamente

$\text{NP} \neq \text{ESPACIO}(n).$

■

Nota (por qué este argumento no decide la inclusión). El razonamiento prueba la *desigualdad* de clases sin decir cuál contiene a cuál — de hecho hoy se desconoce si $\text{NP} \subseteq \text{ESPACIO}(n)$. La técnica (“una clase es cerrada bajo cierto tipo de reducción y la otra no, luego son distintas”) es un patrón clásico para separar clases. El único ingrediente no elemental es el Teorema de Jerarquía de Espacio, que proporciona el lenguaje $A \in \text{ESPACIO}(n^2) \setminus \text{ESPACIO}(n)$; este teorema se demuestra por *diagonalización* (se construye una máquina que en espacio n^2 contradice a toda máquina de espacio lineal). La maniobra de *padding* “traslada” esa separación a un par de lenguajes relacionados por una reducción trivial, exhibiendo el fallo de cierre.

Fin de la Relación 4.

Documento elaborado con fines de estudio. Los resultados son estándar en teoría de la complejidad (Sipser; Papadimitriou; Garey–Johnson); las demostraciones siguen los esquemas habituales de esos textos y la terminología de la asignatura.